

Développer en utilisant une BD

Premiers pas : procédures stockées & JDBC

Géraldine Del Mondo

Département Informatique et Technologies de l'Information

Institut National des Sciences Appliquées - Rouen

geraldine.del_mondo@insa-rouen.fr

Plan...

- 1 Présentation
- 2 Couplage avec les langages de programmation
 - Un autre langage dans du SQL
 - SQL embarqué
- 3 Conclusion

Jusqu'ici, quelle interaction avec la base de données ?

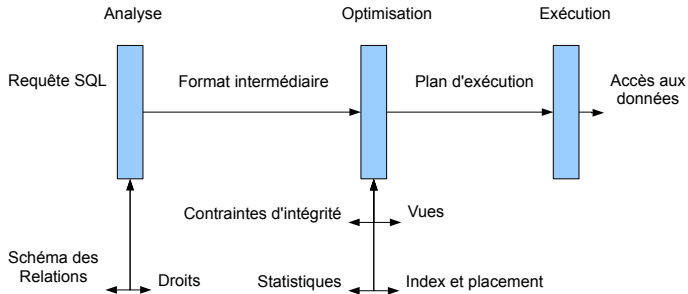
- **Installation / Configuration** du SGBD (PostgreSQL)
- **Création** : base (CREATEDB), utilisateurs (CREATEUSER), ...
- **Connection** à une base de données (psql)

```
psql -h iti-pg.insa-rouen.fr -U grtti viticole
```

- locale ou a distance (option -h)
 - selon un certain utilisateur (option -U)
 - sur une certaine base
- **Déroutement**
 - Edition d'une requête
 - Interprétation : correcte → exécution / incorrecte → ressaisie
 - Affichage du résultat
 - **Déconnection** (\q)

→ **Démo**

Séquencement d'une requête



Insuffisance de SQL

- Structure de programme
- Structure de contrôle
- Structure de données

Solutions

- 1 Les “procédures stockées” : Utiliser un autre langage dans du SQL
- 2 “Embedded SQL” : Plonger SQL dans un langage de programmation

1- Utiliser un autre langage dans du SQL

- Code réalisé dans le cadre de la création de la base de données
- **PL/PGSQL**, PL/SQL, C ..

Avantages

Permet d'optimiser l'exécution de certaines parties d'un projet (procédures pré-compilées)

Inconvénient

Maintenance, dépendant du SGBD (s'exécute sur le moteur de base de données)

Couplage SQL - PL/PGSQL

→ Très similaire au PL/SQL (Oracle)

→ Langage Procédural

- Est utilisé pour créer des fonctions standards et triggers
- Ajoute des structures de contrôle au langage SQL
- Permet d'effectuer des traitements complexes

Base exemple

PARCNAT (idZone VARCHAR(10), type VARCHAR(15),
description VARCHAR(50), superficie REAL, pays VARCHAR(20),
ouvertD DATE, ouvertF DATE, coutInter INTEGER)

ANIMAUX (noeAnim VARCHAR(10), espece VARCHAR(15),
idZone VARCHAR(10), dateNais DATE)

VETO (noVeto VARCHAR(10), nom VARCHAR(15),
prenom VARCHAR(15), etablisement VARCHAR(20),
compEspece VARCHAR(15), dateDispoD DATE, dateDispoF DATE)

PLANNING (noInter VARCHAR(10), noVeto VARCHAR(10),
idZone VARCHAR(10), dateInterD DATE, dateInterF DATE)

Fonction PL/PGSQL

CREATE OR REPLACE FUNCTION nomFonction ($p_1, \dots, p_n$)
RETURNS *Type* **AS** \$\$

DECLARE

- - Déclaration des variables locales

nomVariable *Type* ;

BEGIN

- - Corps de la fonction : requêtes SQL/ structure de contrôle etc.

RETURN résultat ;

END \$\$ LANGUAGE 'plpgsql' ;

EXEMPLE (1)

```
CREATE OR REPLACE FUNCTION increment(i Integer)
RETURNS Integer AS $$
BEGIN

RETURN i + 1;
END $$ LANGUAGE 'plpgsql';
```

Exécution

```
SELECT increment(2);
   increment
-----
           3
(1 ligne)
```

→ Démo

EXAMPLE (2)

```
CREATE OR REPLACE FUNCTION periodeDOuvertureParc  
(zone VARCHAR(10)) RETURNS Integer AS $$
```

```
DECLARE
```

```
nbjours Integer ;
```

```
BEGIN
```

```
select ouvertF - ouvertD into nbjours
```

```
from parcnat
```

```
where zone = idzone ;
```

```
RETURN nbjours ;
```

```
END $$ LANGUAGE 'plpgsql' ;
```

EXEMPLE (2) - exécution

PARCNAT

idZone	type	description	superficie	pays	ouvertD	ouvertF	coutInter
pn1	forêt	joli coin	120.5	France	01/01/2012	12/01/2012	150

```
select periodeD OuvertureParc('pn1');
```

```
periodedouvertureparc
```

```
-----
```

```
11
```

```
(1 ligne)
```

EXEMPLE (3) - travailler sur plusieurs tuples : utilisation d'un curseur

```
CREATE OR REPLACE FUNCTION majoreCout()  
RETURNS SETOF parcnat AS $$
```

```
DECLARE
```

```
monCurseur Refcursor ;
```

```
uneLigne PARCNAT%ROWTYPE ; /* définir ses types : cf. TP */
```

```
BEGIN
```

```
OPEN monCurseur SCROLL FOR SELECT idZone, descrip-  
tion, coutInter * 2 FROM parcnat ;
```

```
...
```

EXEMPLE (3) - utilisation d'un curseur (suite)

...

LOOP

FETCH NEXT FROM monCurseur INTO uneLigne;

IF (uneLigne **IS NULL**) **THEN**

EXIT;

END IF;

RETURN NEXT uneLigne;

END LOOP;

CLOSE monCurseur;

END \$\$ LANGUAGE 'plpgsql';

EXEMPLE (3) - execution

PARCNAT

idZone	type	description	superficie	pays	ouvertD	ouvertF	coutInter
pn1	forêt	joli coin	120.5	France	01/01/2012	12/01/2012	150
pn2	montagne	zone fraiche	100.25	France	10/07/2012	12/08/2012	200
pn3	forêt	une île	50.5	Madagascar	11/08/2012	13/09/2012	400
pn4	savane	zone aride	500.5	Kenya	15/12/2012	13/01/2013	400
pn5	océan	plouf	1500.0	Australie	03/03/2012	15/07/2013	450
pn6	forêt	tropical	600.55	Honduras	03/05/2012	26/11/2012	400

parcnat

```
select majoreCout();  
      majorecout
```

```
-----  
(pn1,"joli coin",300,,,,)  
(pn2,"zone fraiche",400,,,,)  
(pn3,"une île",800,,,,)  
(pn4,"zone aride",800,,,,)  
(pn5,plouf,900,,,,)  
(pn6,tropical,800,,,,)  
(6 lignes)
```

EXAMPLE (3) - utilisation d'un curseur (version simplifiée)

```
CREATE OR REPLACE FUNCTION majoreCout()  
RETURNS SETOF parcnat AS $$
```

```
DECLARE  
monCurseur PARCNAT%ROWTYPE ;
```

```
BEGIN  
  FOR monCurseur IN
```

```
  SELECT idZone, description, coutInter * 2  
  FROM parcnat  
  LOOP
```

```
    RETURN NEXT monCurseur ;  
  END LOOP ;
```

```
RETURN ;  
END $$ LANGUAGE 'plpgsql' ;
```


EXEMPLE (4) - création d'un trigger simple [étape 1]

```
CREATE OR REPLACE FUNCTION monPremierTrigger()  
RETURNS trigger AS $$  
DECLARE
```

```
coutIntervention Integer;
```

Objectif : Insertion ok si coutInter
du nouveau tuple < coutInter du
tuple 'pn5'

```
BEGIN
```

```
select coutInter into coutIntervention from parcnat p  
where p.idZone = 'pn5';
```

```
IF (NEW.coutInter > coutIntervention) THEN  
RAISE EXCEPTION 'cout intervention trop élevé : %, doit être  
inférieur à %', new.coutInter, coutIntervention;  
END IF;
```

```
RETURN NEW;  
END $$ LANGUAGE 'plpgsql';
```

EXEMPLE (4) - création d'un trigger simple [étape 2]

```
CREATE TRIGGER testCoutIntervention  
BEFORE INSERT ON parcnat  
FOR EACH ROW EXECUTE PROCEDURE monPremierTrigger();
```

idZone	type	description	superficie	pays	ouvertD	ouvertF	coutInter
pn5	océan	plouf	1500.0	Australie	03/03/2012	15/07/2013	450

Exécution

```
INSERT INTO PARCNAT  
VALUES
```

```
('pn7', 'océan', 'joli coin coin' ,200 , 'France',  
  '01/01/2013', '12/01/2013',490);
```

ERROR: cout intervention trop élevé : 490,
doit etre inférieur à 450

Pour info : for each row vs. for each statement

- **for each row**

- Appelé pour chaque ligne que l'opération modifie

e.g. un delete qui affecte 10 lignes entraine 10 appels distinct du trigger

e.g. afficher à chaque modif. de ligne un nouveau salaire, l'ancien et la différence entre les 2

- **for each statement**

- opération de groupe, ne s'exécute qu'une fois pour une opération donnée quel que soit le nombre de ligne modifiées

e.g. calculer une moyenne des salaires après une m.a.j

→ Quand on peut il est plus performant d'utiliser " for each statement" mais ... OLD et NEW ne sont pas disponible pour ce type de trigger
e.g. déclencheur 'Instead of' : for each row, il faut avoir accès à l'ancienne/nouvelle valeur

EXEMPLE (5) - Syntaxe de suppression fonction/trigger

```
DROP TRIGGER testCoutIntervention ON parcnat ;
```

```
DROP FUNCTION monpremiertrigger() ;
```

2 - “Embedded SQL” : Plonger SQL dans un langage de programmation

- Technique de pré-compilation (éventuellement)
- Normalisée SQL1 (PL/1, Cobol, Pascal, Fortran), SQL2 (**Java**, **C**, Ada)

Avantages

Indépendant du SGBD

Inconvénient

Maintenance : deux systèmes de types / styles de programmation différents

Fonctionnalités du couplage SQL - langage de programmation

- Procéduralité du langage source
- "Indépendance du SGBD"
- Gestion des transactions
- Gestion des erreurs
- SQL dynamique (construction de l'ordre à la volée)

Introduction à JDBC (N. Malandain)

Java DataBase Connectivity

- bibliothèque d'interfaces et de classes
- communication avec un SGBDR
- manipulation de requêtes SQL

Drivers

Il existe différents drivers JDBC pour différentes bases de données

Création/Fermeture de la connexion

CHARGEMENT DU DRIVER (FACULTATIF DEPUIS LA VERSION 4 DE JDBC)

```
Class.forName(''classedriver'');
```

→ PostgreSQL driver : `org.postgresql.Driver`

OUVERTURE DE LA CONNEXION

```
Connection connection;  
connection=DriverManager.getConnection("UrlDeLaBaseDeDonnées","login","passwd");
```

→ url de la base :

```
jdbc:postgresql://iti-pg.insa-rouen.fr:5432/nomBase
```

FERMETURE DE LA CONNEXION

```
connection.close();
```


Requêtes

CRÉATION D'UN OBJET STATEMENT

```
Statement monStatement = connection.createStatement();
```

- cet objet permet l'exécution de requêtes SQL
- un seul objet ResultSet lui est associé

EXÉCUTION DE REQUÊTES SUR L'OBJET STATEMENT

- `int executeUpdate("requete modification (insert/update/delete)");`
- `ResultSet executeQuery("requete de consultation (select)");`

UTILISATION DE RESULTSET

- `boolean next(), boolean previous()` // positionnement sur les tuples
- `leType getLeType(String nomColonne)` // récupération des valeurs

EXAMPLE Suppression tuple

```
public static void main(String[] args) throws SQLException,
ClassNotFoundException {
    //Connexion
    Class.forName("org.postgresql.Driver");
    connection = DriverManager.getConnection("
        jdbc:postgresql://iti-pg.insa-rouen.fr:5432/zoo","host","mdp");

    Scanner sc = new Scanner(System.in);

    //Exemple suppression tuple
    System.out.println("Quel tuple de VETO supprimer ? (id)");
    String id = sc.nextLine();

    Statement st = connection.createStatement();
    st.executeUpdate("DELETE FROM VETO WHERE noVeto='"+id+"'");

    st.close()

    //Deconnexion
    connection.close();
}
```

EXEMPLE Parcours ResultSet

```
public static void main(String[] args) throws SQLException,
ClassNotFoundException {
    //Connexion
    connection = DriverManager.getConnection(".....

    //Exemple ResultSet
    Statement st = connection.createStatement();
    ResultSet laTable = st.executeQuery("SELECT * FROM PLANNING");

    while (laTable.next()){
        id = laTable.getString("nointer");
        idV = laTable.getString("noVeto");
        idZ = laTable.getString("idzone");
        dated = laTable.getString("dateinterd");
        dateF = laTable.getString("dateinterf");

        System.out.println(String.format("%s|%s|%s|%s|%s"),id,idV,idZ,
                                         dated, dateF);
    }
    st.close()

    //Deconnexion
    connection.close();
}
```

Statement / PreparedStatement

CRÉATION D'UN OBJET PREPAREDSTATEMENT

- Ce type d'objet est particulièrement adapté pour une requête qui va être exécutée plusieurs fois mais avec des paramètres potentiellement différents.
- Le SGBD prépare la requête lors de la création du PreparedStatement (jusqu'au plan d'exécution).
- Il faut associer les paramètres manquants de la requête avec les valeurs choisies, puis exécuter la requête.

Exemple → DÉMO

```
PreparedStatement especeAnim = connection.prepareStatement(  
    "SELECT noeAnim FROM ANIMAUX where espece=?");  
  
especeAnim.setString(1, "MACAO");  
ResultSet laTableDesPerroquets = especeAnim.executeQuery();  
...  
especeAnim.setString(1,"LION");  
ResultSet laTableDesFelins = especeAnim.executeQuery();  
...  
especeAnim.close();
```

MetaData

INFORMATIONS MÉTABASE

Pour accéder aux information de la métabase, il est possible d'utiliser en particulier deux interfaces : DatabaseMetaData et ResultSetMetaData.

EXEMPLE

```
...  
DatabaseMetaData meta = connection.getMetaData();  
  
String utilisateur = meta.getUserName();  
  
System.out.println("user = "+utilisateur);  
...
```

Appel procédure stockée : interface CallableStatement

- Appel de procédure avec ou sans paramètre
- Appel de fonction avec ou sans paramètre

EXEMPLE APPEL FONCTION

```
...
CallableStatement st =
    conn.prepareCall("{? = call maFonction(?,?)}");
...
```

Il faut ensuite définir le(s) type(s) de retour (méthode `registerOutParameter`) et les paramètres de la fonction (méthode `setNOM-DU-TYPE-DU-PARA`).

```
st.registerOutParameter(1, typeSortie);
st.setType(2, nomParametre1);
st.setType(3, nomParametre2);
st.execute();
```

```
// st.gettypeSortie(1) pour recuperer le resultat
```



EXEMPLE appel procédure stockée

EXEMPLE APPEL PROCÉDURE STOCKÉE

```
public void calculSalaireVeto(int coutParJour, String idVeto)
throws SQLException{

    CallableStatement st =
        conn.prepareCall("{? = call calculSalaire(?,?)}");

    st.registerOutParameter(1, java.sql.Types.INTEGER);
    st.setInt(2, coutParJour);
    st.setString(3, idVeto);
    st.execute();

    System.out.println(String.format("Salaire veto %s : %d",
                                     idVeto, st.getInt(1)));
}
```

Remarque :

- Si l'on est susceptible de récupérer un ensemble de tuples alors il n'y a pas de sortie à déclarer lors du *prepareCall*

→ ResultSet

Gestion des transactions en Java



TRANSACTION (RAPPEL)

Ensemble d'instructions effectuées ensembles pour assurer la cohérence des données de la base

MÉTHODE

```
connection.setAutoCommit(false);
try {
    // instructions java et/ou SQL de la transaction

    // validation de la transaction
    connection.commit();
}
catch (SQLException e) {
    // code de traitement
    connection.rollback();
}
catch (Exception e) {
    // code de traitement
    connection.rollback();
}
connection.setAutoCommit(true);
```


EXEMPLE Gestion de transactions

Objets à créer :

A
a : PK b : FK

B
b : PK a : FK

- Un objet ne peut être créé avant l'autre, il faut :
 - commiter seulement après les deux créations
 - que les CIs ne soient vérifiées qu'au moment du commit

Critique du SQL embarqué

Inconvénients

- Connaissance de deux langages : langage hôte et SQL
- Langage de type non uniforme (type langage hôte versus type SQL)
- Impedance mismatch : unitaire (langage hôte) vs. ensembliste (SQL)
- Accès au niveau attribut
- Optimisation faible

Avantage

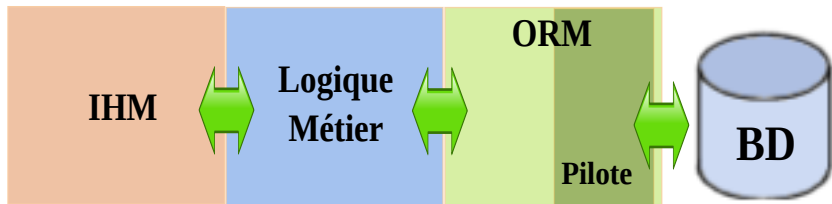
- Indépendant du SGBD

Conclusion

- Norme internationale ISO
 - SQL1 : SQL86 : SQL1 niveau 1 = 100 pages / SQL89 : SQL1 niveau 2 = 120 pages, 94 mot-clés
 - SQL2 : SQL92 : 520 pages, 224 mot-clés
 - SQL3 : > 1000 pages
- Evolution d'un simple langage d'accès à une BD relationnelle (SQL1) vers un langage de programmation BD (SQL3 ?)
- Langage complexe non accessible à un utilisateur final
- Dualité : couplage langage de programmation classique + BD vs. langage dédié applications BD

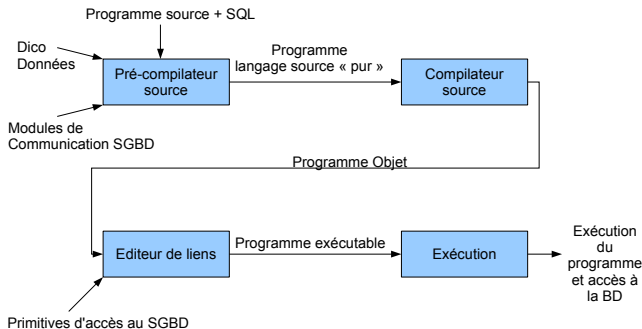
→ ORM...

ORM & cie



COMPLÉMENT : COUPLAGE SQL-C

En C comment ça marche ?



- Pré-processeur **ecpg**, extension fichier C avec SQL embarqué : **.pgc**
 - **ecpg** mon fichier.pgc
- + compilation classique en C avec au moins la librairie **libecpg**

Objectif : communication entre le programme et le(s) SGBD(s)

- DÉCLARATION

```
EXEC sql BEGIN DECLARE SECTION  
  int numAnim;  
  char esp[15];  
  char idZ[10];  
  date nais;  
EXEC sql END DECLARE SECTION;
```

- UTILISATION

- Dans les instructions du langage source → NOM_VARIABLE (e.g. numAnim)
- Dans les requêtes SQL → :NOM_VARIABLE (e.g. :numAnim)
- Accès aux données de la base au niveau "attribut"

Manipulation des données

- INTERROGATION

- un seul n-uplet : SELECT
- un ensemble de n-uplet : mécanisme de curseur

- MISE À JOUR

- Insertion d'un n-uplet : INSERT INTO
- Suppression d'un ensemble de n-uplet : DELETE
- Modification d'un ensemble de n-uplet : UPDATE

Mécanisme de curseur

Problème :

- Requête SQL (SELECT) : Ensemble de n-uplet
- Langage hôte : n-uplet par n-uplet (pas de notion d'ensemble)

Solution :

- Ouverture d'un "fichier logique séquentiel"
- Gestion d'un pointeur (parcours du fichier)
- Fermeture du fichier

Gestion d'un curseur

- DÉCLARATION

EXEC SQL DECLARE leCurseur CURSOR
FOR <Requête SQL>

- MANIPULATION

- **Ouverture** : exécution de la requête et positionnement du "début" du curseur (avant le premier n-uplet)
→ EXEC sql OPEN leCurseur
- **Lecture n-uplet suivant**
→ EXEC sql FETCH leCurseur INTO...
S'il existe (exemple de condition d'arrêt : sinon SQLCODE = 100)
- **Fermeture**
→ EXEC sql CLOSE leCurseur

Contrôle de l'exécution des requêtes BD

- **SQLCA** -Communication Area- : contient les informations de contrôle venant du SGBD à l'issue de l'exécution de la requête SQL
 - SQLSTATE : statut d'exécution de la dernière requête BD (par classe) (`sqlca.sqlstate`)
 - 00000 : correct
 - 01XYZ : avertissement
 - 02XYZ : pas de données
 - ...
 - SQLERRD : contient l'OID (/nombre) de la ligne(s) traitée(s) (`sqlca.sqlerrd[1]` / `sqlca.sqlerrd[2]`)
 - fonction SQLERRM : obtention du message d'erreur associé (SQLCODE) (`sqlca.sqlerrm.sqlerrmc`, et longueur du message dans `sqlca.sqlerrm.sqlerrm`)
 - ...

Gestion des erreurs

- DÉFINITION DE LA ZONE SQLCA

EXEC sql include SQLCA

- GESTION DES ERREURS

- IF SQLCODE == 0 THEN *ok*
- IF SQLCODE < 0 THEN *erreur à gérer*
- IF SQLCODE == 100 *plus de n-uplet à lire*
- Test par le système (exception)
 - EXEC sql WHENEVER SQLWARNING continue
 - EXEC sql WHENEVER SQLERROR SQLPRINT
 - ...

EXEMPLE en C

*/*DÉCLARATION DES VARIABLES*/*

- Relation ANIMAUX (noeAnim, espece)
- Programme de gestion (partie DÉCLARATION)

```
#include <stdio.h>
#include <string.h>
#include <sqlca.h>
```

```
int main (int argc , char *argv []) {
```

```
EXEC SQL BEGIN DECLARE SECTION;
```

```
    char UID[20];
    char PWD[20];
    char base [20];
    int numAnim;
    char esp[15];
```

```
    /*    int arrêter; */
```

```
EXEC SQL END DECLARE SECTION;
```

```
/* arreter = 0; */
```

EXEMPLE (suite)

```
/*GESTION DES ERREURS */  
EXEC SQL INCLUDE sqlca ;  
EXEC SQL WHENEVER sqlerror sqlprint ;  
  
/*CONNEXION À LA BASE*/  
strcpy (base, "tcp :postgresql ://asi-pg.insa-rouen.fr :5432/Zoo" );  
  
printf (" User et password ? " );  
scanf ("%s%s",UID,PWD);  
  
EXEC SQL CONNECT TO :base USER :UID USING :PWD ;
```

EXEMPLE (suite)

```
/*DÉCLARATION CURSEUR*/
```

```
EXEC SQL DECLARE monCurseur CURSOR FOR  
SELECT noeAnim, espece  
FROM ANIMAUX  
WHERE espece != 'ALPAGA';
```

EXEMPLE (suite - version 1)

```
/*TRAITEMENT : PARCOURS DE LA TABLE - GESTION DU CURSEUR */  
EXEC SQL OPEN monCurseur;  
  
EXEC SQL WHENEVER NOT FOUND DO BREAK;  
while (1)  
{  
    EXEC SQL FETCH monCurseur INTO :numAnim, :esp;  
  
    printf ("numéro animal = %s,  
           espèce animal = %s", numAnim, esp);  
}  
EXEC SQL WHENEVER NOT FOUND CONTINUE;  
  
EXEC SQL CLOSE monCurseur;  
  
EXEC SQL DISCONNECT;  
return EXIT_SUCCESS;  
} /* FERMETURE DE LA FONCTION MAIN */
```


EXEMPLE (suite - version 2)

```
/*TRAITEMENT : PARCOURS DE LA TABLE - GESTION DU CURSEUR */
```

```
EXEC SQL OPEN monCurseur ;
```

```
/ * ARRETER = 0 ; */
```

```
do {
```

```
    EXEC SQL FETCH monCurseur INTO :numAnim, :esp ;
```

```
    if (sqlca.sqlerrd[2] == 1) {
```

```
        printf (" numéro animal = %s,
```

```
                espèce animal = %s", numAnim, esp) ;
```

```
    } else {arreter = 1 ;}
```

```
    } while (arreter != 1) ;
```

```
EXEC SQL CLOSE monCurseur ;
```

```
EXEC SQL DISCONNECT ;
```

```
return EXIT_SUCCESS ;
```

```
} /* FERMETURE DE LA FONCTION MAIN */
```

SQL Dynamique - Immédiat

DÉCLARATION D'UNE VARIABLE POUR STOCKER LA REQUÊTE

```
EXEC SQL BEGIN DECLARE SECTION ;  
const char* RequeteSource="ma requête sql" ;  
EXEC SQL END DECLARE SECTION ;
```

EXÉCUTION DE LA REQUÊTE

```
EXEC SQL EXECUTE IMMEDIATE :RequeteSource ;
```

ATTENTION

EXECUTE IMMEDIATE ne peut être utilisé que pour des ordres SQL qui ne retournent pas de données (e.g. LDD) sinon il faut utiliser du *différé*.

SQL Dynamique - Différé (1)

DÉCLARATION D'UNE VARIABLE POUR STOCKER LA REQUÊTE

```
EXEC SQL BEGIN DECLARE SECTION ;  
const char* RequeteSource="ma requête sql" ;  
char esp[15] ;  
EXEC SQL END DECLARE SECTION ;
```

LECTURE DE LA REQUÊTE

Par exemple ma requête sql = 'select espece from ANIMAUX where noeanim='a1-a1' ;

COMPILATION DE LA REQUÊTE

```
EXEC SQL PREPARE RequeteObjet FROM :RequeteSource ;
```

EXÉCUTION DE LA REQUÊTE

```
EXEC SQL EXECUTE RequeteObjet INTO :esp ;
```

SQL Dynamique - Différé (2)

DÉCLARATION DU NOM DE LA REQUÊTE / REQUÊTE COMPILÉE

```
EXEC SQL BEGIN DECLARE SECTION ;  
const char* RequeteSource= "ma requête sql" ;  
EXEC SQL END DECLARE SECTION ;
```

LECTURE DE LA REQUÊTE

Ici par exemple ma requête sql = 'delete from ANIMAUX where
noeAnim = ? ;'

COMPILATION DE LA REQUÊTE

```
EXEC SQL PREPARE RequeteObjet FROM :RequeteSource ;
```

EXÉCUTION DE LA REQUÊTE

```
EXEC SQL EXECUTE RequeteObjet USING 'a1-a1' ;
```

SQL Dynamique - Différé (suite)

SORTIE PROPRE

EXEC SQL DEALLOCATE PREPARE RequeteObjet ;

Remarques et synthèse

- **Usage du curseur** : Dès qu'une requête retourne plusieurs tuples au résultat $\Rightarrow$ CURSEUR
- **SQL Dynamique** : Une commande **EXECUTE** peut avoir une clause INTO, une clause USING, les deux, ou aucune.

EXECUTE IMMEDIATE ne peut être utilisé que pour des ordres SQL qui ne retournent pas de données,

sinon il faut utiliser du *différé* et la clause EXEC SQL PREPARE

- **NB** : Pensez à entériner l'exécution des requêtes : **EXEC SQL COMMIT ;**